Proteção de Rotas em JavaScript

Parte 2 — Middleware · Rotas Protegidas · Testes

#JAVASCRIPT #EXPRESS #MIDDLEWARE #JWT

Made with GAMMA

O que é um Middleware?

Fluxo Visual

Requisição → Middleware → Controller → Resposta

Como funciona

Um middleware é uma função que roda **ANTES** do controller. Usamos para verificar se o token é válido antes de permitir acesso à rota.

- Se o token for inválido, o middleware barra a requisição e nem chega ao controller.

Made with GAMMA

Middleware de Autenticação

middlewares/authMiddleware.js

```
const autenticar = (req, res, next) => {
 const token = req.headers["authorization"]?.split(" ")[1];

 if (!token) return res.status(401).json({ mensagem: "Token não fornecido" });

 try {
 req.usuario = jwt.verify(token, process.env.JWT_SECRET);
 next(); //
```

Made with GAMMA

Middleware de Admin

Restringindo acesso por perfil

```
const apenasAdmin = (req, res, next) => {
  if (req.usuario.role !== "admin") {
    return res.status(403).json({ mensagem: "Acesso negado" });
  }
  next();
};
```

✅ role: "admin"

Acesso liberado — o middleware chama `next()` e a requisição segue normalmente para o controller.

❌ role: "user"

403 Forbidden — mesmo com token válido, o usuário não tem permissão de admin e a requisição é bloqueada.

Made with GAMMA

Aplicando no server.js

Rotas públicas vs Rotas protegidas

🔓 PÚBLICA (sem token)

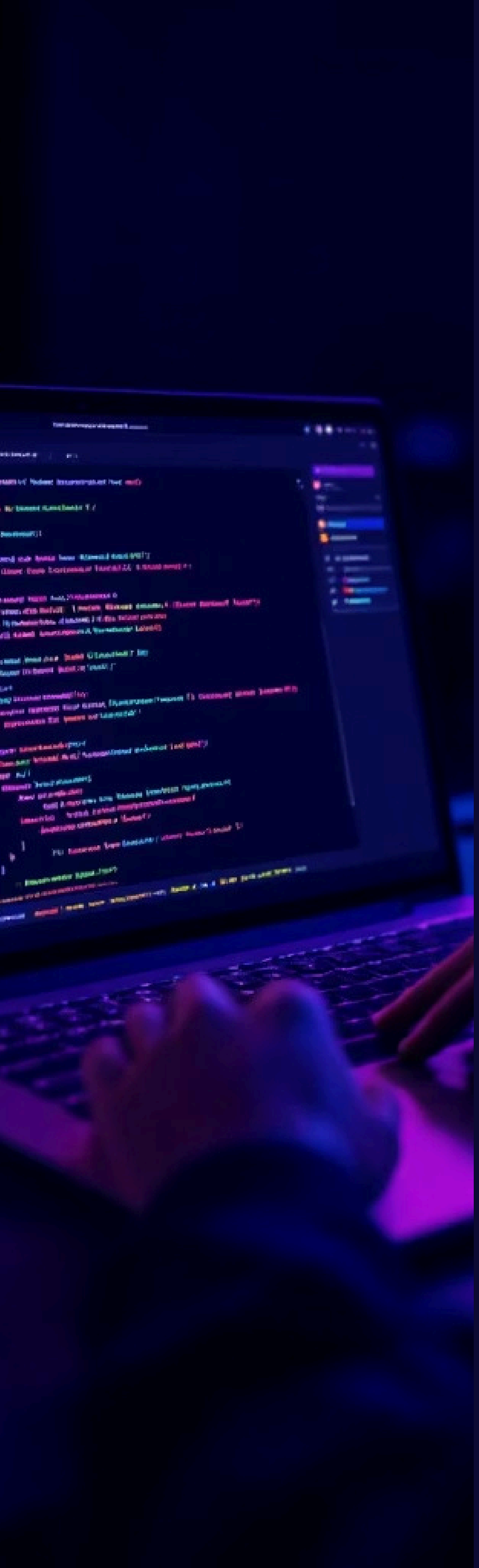
```
// Login não exige autenticação
app.use("/auth", authRoutes);
```

🔒 PROTEGIDA (exige token)

```
// Middleware aplicado a tudo abaixo
app.use(autenticar);
app.use("/produtos", produtoRoutes);
app.use("/usuarios", usuarioRoutes);
app.use("/categorias", categoriaRoutes);
```

⚠️ **A ordem importa!** Rotas públicas devem vir ANTES do middleware.

Made with GAMMA



Testando no Thunder Client

Como testar no Thunder Client — Passo a passo

1

POST /auth/login

```
Body:
{
  "email": "admin@fazenda.com",
  "senha": "123456"
}
```

2

Copie o token da resposta

```
{
  "token": "eyJhbGci..."
}
```

3

Adicione o header

Na próxima requisição → aba **Headers**

```
Authorization: Bearer eyJhbGci...
```

4

Resultados possíveis

- ✅ **200 OK** → token válido, acesso liberado
- ❌ **401 Unauthorized** → token inválido ou ausente
- ❌ **403 Forbidden** → token válido mas sem permissão

Made with GAMMA

Erros Comuns

Erro	Causa	Solução
`401 Unauthorized`	Token não enviado ou inválido	Verificar header `Authorization`
`403 Forbidden`	Token válido mas sem permissão	Verificar `role` do usuário
`TokenExpiredError`	Token expirou	Fazer login novamente
`JsonWebTokenError`	Token malformado	Verificar se está enviando corretamente
Senha não confere	Comparando texto puro com hash	Usar `bcrypt.compare()`

Made with GAMMA

Fluxo Completo

Tudo junto — do cadastro ao acesso protegido

CADASTRO

`POST /usuarios`

- Senha criptografada com `bcrypt`
- Salvo no banco

LOGIN

`POST /auth/login`

- `bcrypt.compare()`
- `jwt.sign()`
- Token retornado

ACESSO PROTEGIDO

`GET /produtos + token no header`

- Middleware valida
- Controller responde

Made with GAMMA

Boas Práticas em Produção

🔐 JWT_SECRET no .env

`JWT_SECRET` sempre no `.env` — **nunca** no código.

⏰ Token com expiração

Token com expiração definida (`expiresIn: "1d"`).

🔄 Use HTTPS

Use HTTPS — o token trafega no header e precisa de conexão segura.

🚫 Payload sem dados sensíveis

Nunca exponha dados sensíveis no payload do JWT.

Made with GAMMA

Resumo Final

Tudo que você precisa lembrar

✅ bcrypt.hash()

Criptografa senha

✅ bcrypt.compare()

Verifica senha

✅ jwt.sign()

Gera token

✅ jwt.verify()

Valida token

✅ Middleware

Protege rotas

✅ next()

Libera acesso

✅ Authorization: Bearer

Header correto

✅ .env

Guarda o segredo

Made with GAMMA